Abhay Kiran M
2611006200002

RTOS Assignment - 2

(1) Bounded Buffer :-

Consider two processes which are called producer and consumer respectively. The producer is a cyclic process and each time it goes through its cyclic cycle it produces a certain portion of information that has to be processed by the consumer. The consumer is also cyclic process and each time it goes through its cycle it can process the next portion of information as has been produced by the producer.

This producer consumer relationship applies a one way communication channel. We also now assume in our applications that the information produced & need not be consumed immediately so we can use a queue as a buffer to hold the information until it is requested by consumers.

We can again understand the need for a buffer using an example if a villager has to instantly eat those carrots his farm produces or they will go bad if he tries to store them somewhere. The carrots will be wasted and his farm will remain efficient if it produces the exact number required by the villager (consumer). A buffer can solve this by acting as storage if more carrot produced than villager can eat it goes into buffer and when there is less to eat he can pull from the buffer to ~~a~~ satisfy his hunger. ~~so both~~

The synchronization constraints :-

- While an item is being added or removed from the buffer the buffer is an inconsistent state. Therefore threads must have exclusive access to the buffer
- If a consumer thread arrives while the buffer is empty, it blocks until a producer adds a new item.

Since our buffer is bounded additional constraint

- If a producer arrives when the buffer is full, it blocks until a consumer removes an item.

A simple ~~way would be (to implement a buffer)~~

~~if items == buffer size~~
~~block().~~

En → unbounded buffer :-

```
mutex = semphore(1)     // exclusive access to buffer
items = semephore(0)    // num of items in buffer

producer :-
event = wait For Event()
mutex.wait()
    buffer.add(event)
mutex.signal()
items.signal()

consumer
items.wait()
mutex.wait()
```

classmate
Date
Page

event = buffer.get()
mutex.signal()
mutex.process()

when items is positive it is the number of items in the buffer, when it is negative its magnitude is the number of consumers waiting. A consumer cannot enter a buffer but a producer has signalled items, so it can never remove from empty buffer.

Now for an unbounded buffer you must write

if items >= buffer size : block(), as semaphore value cannot be used. Instead add a second counting semphore

mutex = semaphore(1)
items = semaphore(0)
spaces = Semaphore(buffer.size())

event = wait for Event()
spaces.wait()
mutex.wait()
buffer.add(event)
mutex.signal()
items.signal

items.wait()
mutex.wait()
event = buffer.get()
mutex.signal()
spaces.signal()
event.process()

The solution is symetric locks and spaces are binary semaphores and their sem always equals the buffer size. In order to avoid deadlock producers and consumers check availability before getting the mutex. The release mutex before signalling for best performance. Spaces is a multiplex during how many items can be inside the buffer and items is a signalling semaphore.

The unbounded buffer was originally penned by dykstra to solve the problem of a computer ~~punching~~ card producing card images faster in burst than a card punch could print them. ~~The~~

" The unbounded buffer is the standard way to decouple two activities that run at different speeds".

The bound matters because without it faster producer exhausts memory and blocking the producer is form of back-pressure that keeps the system stable.

(2) Readers writers problem :-

The basic readers writer solution is easy and deadlock free but it lets writers starve. The real content of the problem is choosing who gets priority and accepting that the other side may wait longer.

Problem statement:

The problem pertains to any situation where a data structure, database or file system is read and modified by concurrent threads. A reader must not see a modification half-done, so writing must exclude everyone.

1. Any number of readers can be in the critical section simultaneously.

2. Writers must have exclusive access to the critical section.

(i) Writer priority solution:

If writers make time-critical updates, it is better that once a writer arrives no new reader enters until all writers are done.

```
readSwitch = LightSwitch()
writeSwitch = LightSwitch()
noReaders = Semaphore(1)
noWriters = Semaphore(1)

# readers
noReaders.wait()
    readSwitch.lock(noWriters)
noReaders.signal()

<critical section>

readSwitch.unlock(noWriters)

writeSwitch.lock(noReaders)
```

noWriters.wait()
noWriters.signal()
writeSwitch.unlock(noReaders)

A reader must lock writer's but not other readers so an arriving writer can block noReaders and make all later readers queue. The class object's effect of writeSwitch is that many writers can queue on noWriters and pass through one after another while noReaders stay locked.

Readers writers at the first problem where correctness is not enough. All three solutions are safe and deadlock free they differ only in waits. This forces the designer to state a fairness policy explicitly.

Starvation :-

The basic solutions can't deadlock because no threads wait on a semaphore while holding another in a way that prevents its own wakeup. But a writer can starve. If readers keep arriving so that a new one enters before the last current one leaves the room is never empty and the writer waits forever. Such a program might work as long as the demand is low and then degrade gradually as the load rises. This is exactly kind of bug that passes basic testing and fails in field

No starve solution:

```
readSwitch = lightSwitch()
roomEmpty = semaphore(1)
turnstile = semaphore(1)

turnstile.wait()
  roomEmpty.wait()
turnstile.signal()
roomEmpty.signal()

turnstile.wait()
turnstile.signal()
readSwitch.lock(roomEmpty)

readSwitch.unlock(roomEmpty)
```

Another that writers with readers are stuck blocks on roomEmpty while holding the turnstile. New readers queue at the turnstile. When the last reader decides ~~to~~ it signals roomEmpty and the waiting writer enters before any queued reader can pass. It guarantees atleast one writer proceeds but readers can still starve while writers are queued.

(8) The dining philosophers problem

Five philosophers sit at a round table with five plates, five forks and a bowl of spaghetti. Each repeats:

```
while True:
    think()
    get_forks()
    eat()
    put_forks()
```

The forks are resources a thread must hold exclusively to make progress. What makes the problem interesting is that a philosopher needs two forks, each shared with a neighbour. Philosophers and forks are numbered 0 to 4 and left(i) = i and right(i) = (i + 1) % 5

Solution must satisfy 4 constraints

1. Only one philosopher can hold the fork at a time
2. Deadlock must be impossible
3. Philosopher must not starve waiting for a fork.
4. More than one philosopher must be able to eat at the same time.

The fourth constraint is the efficiency requirement; without it a single eater around the whole table would be a trivial answer. But must be correct. If philosopher could keep a fork forever, nothing could prevent the neighbours from starving

(i) Solution : footman dut dinis

One fork is left on the table. The fork has two neighbours each already holding another fork, so either neighbour can pick up and eat.

dut is multiplex called footman initialised to 4:

```
def get_forks(i):
    footman.wait()
    fork[right(i)].wait()
    fork[left(i)].wait()

def put_forks(i):
    fork[right(i)].signal()
    fork[left(i)].signal()
    footman.signal
```

This also prevents starvation. Suppose you are blocked waiting for your right fork while both neighbours eat. Your right neighbour must eventually put it down because it terminates and since you are the only thread waiting for the fork you get it next. The same argument applies to the left fork, so this at the table is bounded.

(ii) Solution 2: left hand philosopher.

Assume at least one is a lefty, deadlock is impossible. This can be proven by contradiction. Deadlock requires all just to hold one fork and wait for the other. If philosoper 'i' is a lefter she holds her left fork and waits for her right. so her right neighbour must hold his left fork and wait for his right so he is also a lefter.

What this proof shows in general terms is that deadlock needs a cycle of waits and one thread acquiring in a different order is enough to break the cycle. Extended to many locks this becomes famous engineering rule of always acquiring locks in the same order.

(iii) Solution 3: Starve the philosophers

This keeps a state per philosopher and a semaphore per philosopher.

```
state = ['thinking'] * 5
sem = [semaphore(0) for i in range(5)]
mutex = Semaphore(1)

def get_fork(i):
    mutex.wait()
    state[i] = 'hungry'
    test(i)
    mutex.signal()
    sem[i].wait()
```

```
def put_fork(i):
    mutex.wait()
    state[i] = 'thinking'
    test(right(i))
    test(left(i))
    mutex.signal()

def test(i):
    if state[i] == 'hungry' and state[left(i)] != 'eating' and state[right(i)] != 'eating':
        state[i] = 'eating'
        sem[i].signal()
```

A philosopher acts only when hungry and neither neighbour is eating. If forks are free it proceeds at once otherwise blocks its own semaphore until a neighbour finishes and runs 'test' on it. Deadlock is impossible because the only semaphore shared between philosophers is mutex, and no thread waits while holding it.

Dining philosophers is a standard model of deadlock from circular waiting. It shows that a solution can be locally responsible for every thread and still fail globally, and that the cure is structural: change the shape of who-waits-for-who rather than adding more locks. It also shows that deadlock freedom and starvation-freedom are separate properties that must each be proved.

(4) The sleeping barber problem.

The shop has a waiting room with n chairs and a barber with one barber chair.

- If there are no customers the barber goes to sleep.
- If a customer enters and all chairs are occupied the customer leaves.
- If the barber is busy but chairs are free the customer sits in a free chair.
- If the barber is asleep, customer wakes him.

Ex:-

(i) Sol (i)

```
customers = 0
mutex = semaphore (1)
customer = semaphore (0)
barber = semaphore (0)

mutex.wait()
if customer == n+1)
  mutex.signal()
  balk()
customers += 1
mutex.signal()

customer.signal()
barber.wait()
getHairCut()

mutex.wait()
customers -= 1
mutex.signal
```

customer.wait()
barber.signal()
cutHair()

The count is n+1 because the shop full when n customers are waiting and one in the barber chair. A customer who finds the shop full releases the mutex before balking otherwise the mutex would be held forever. Everyone else signals customer and waits on barber

The critical difficulty involved in the problem according is once the barber checks for customers, finds none is about to sleep when a customer arrives and tries to wake him. If sleeping and waking occurs at the same time and checks the wake up would be lost and both would wait forever. The semaphore solution avoids this because a semaphore remembers a signal that occurs before wait. "If thread A gets to the signal just then the value of the semaphore will be incremented However the barbers customer.wait() and a customers customer.signal() interleave, the count of pending customers is not lost

The customer semaphore is doing double duty Its value is the number of customers waiting for service and blocking on its barbers sleep. The barber semaphore lets exactly one customer proceed per haircut which enforces the security one thread in get Hair at a time.

A fundamental weakness with the solution synchronizes start of the haircut but not the end. nothing stops a customer from waiting get Haircut and leaving while the barber is still in at Hair.

Adding a second rendezvous after the barrier would close that gap.

The barbershop is the model for every server that should consume no resource while idle and respond immediately when work arrives. It combines three rules in one problem: a bounded waiting room, a sleeping server that must never miss a wake-up, and admission control, where arrivals are rejected rather than queued without a limit.

Barbara Alain Bazzard